

1 solving congruences

$\mathbb{Z}_m$ denotes the set of nonnegative integers less than m (the remainders). we recall that modular arithmetic “preserves” addition and multiplication, so we can define arithmetic operations restricted to this set.

1.1 solving linear congruences

a linear congruence is an equation of the form $ax \equiv b \pmod{m}$. solving for addition is easy because we can just use subtraction (additive inverse).

example: solve $a + 8 \equiv 2 \pmod{11}$. we can subtract 8 from both sides (or add -8 , which is congruent to 3 modulo 11).

$$a + 8 - 8 \equiv 2 - 8 \pmod{11}$$

$$a \equiv -6 \pmod{11}$$

$$a \equiv 5 \pmod{11}$$

however, for multiplication ($ax \equiv b$), we cannot simply “divide”. we need to find the **multiplicative inverse** of a modulo m . we are looking for a value $|(a)$ such that $a |(a) \equiv 1 \pmod{m}$.

1.2 extended euclidean algorithm

bézout’s theorem states that if $\gcd(a, m) = 1$ (they are coprime), there exist integers s and t such that:

$$sa + tm = 1$$

if we look at this modulo m , the tm term disappears ($tm \equiv 0$), leaving:

$$sa \equiv 1 \pmod{m}$$

this means s is the multiplicative inverse of a !

we can find s and t using the **extended euclidean algorithm**.

1.2.1 example: $57x \equiv 5 \pmod{98}$

to solve this, we first need the inverse of 57 modulo 98. we use the extended euclidean algorithm on 98 and 57.

we want to find s, t such that $98s + 57t = 1$. (skipping the table steps for brevity, but the algorithm works backwards from the remainders).

result: $98 \cdot (-25) + 57 \cdot (43) = 1$. this means $57 \cdot 43 \equiv 1 \pmod{98}$. so, the inverse is **43**.

now we multiply both sides of the original equation by 43:

$$57x \cdot 43 \equiv 5 \cdot 43 \pmod{98}$$

$$(57 \cdot 43)x \equiv 215 \pmod{98}$$

$$1 \cdot x \equiv 19 \pmod{98}$$

so, $x \equiv 19 \pmod{98}$.

1.3 chinese remainder theorem (crt)

this theorem allows us to solve systems of congruences with different moduli, provided the moduli are pairwise coprime.

theorem: let $m_1, m_2, \dots, m_n$ be pairwise coprime positive integers. the system:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

...

$$x \equiv a_n \pmod{m_n}$$

has a **unique solution** modulo $m = m_1 m_2 \dots m_n$.

1.3.1 construction algorithm

to find the solution x :

1. let $m = m_1 m_2 \dots m_n$.
2. let $M_k = \frac{m}{m_k}$ (product of all moduli except m_k).
3. let y_k be the inverse of M_k modulo m_k (so $M_k y_k \equiv 1 \pmod{m_k}$).
4. compute sum: $x = \sum_{k=1}^n a_k M_k y_k$.

1.3.2 example (sunzi suanjing)

solve the system:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

here, $m = 3 \cdot 5 \cdot 7 = 105$.

- **for k=1 ($m_1 = 3, a_1 = 2$):**
 - $M_1 = 5 \cdot 7 = 35$.
 - inverse of 35 mod 3? $35 \equiv 2 \pmod{3}$. inverse of 2 mod 3 is 2 ($2 \cdot 2 = 4 \equiv 1$). so $y_1 = 2$.
- **for k=2 ($m_2 = 5, a_2 = 3$):**
 - $M_2 = 3 \cdot 7 = 21$.
 - inverse of 21 mod 5? $21 \equiv 1 \pmod{5}$. inverse of 1 is 1. so $y_2 = 1$.
- **for k=3 ($m_3 = 7, a_3 = 2$):**
 - $M_3 = 3 \cdot 5 = 15$.
 - inverse of 15 mod 7? $15 \equiv 1 \pmod{7}$. inverse of 1 is 1. so $y_3 = 1$.

final calculation:

$$\begin{aligned} x &= a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3 \\ &= 2(35)(2) + 3(21)(1) + 2(15)(1) \\ &= 140 + 63 + 30 \\ &= 233 \end{aligned}$$

convert to modulo 105: $233 \equiv 23 \pmod{105}$. **solution: 23.**

1.4 fermat's little theorem

theorem: if p is prime and a is not divisible by p , then:

$$a^{p-1} \equiv 1 \pmod{p}$$

this also means $a^p \equiv a \pmod{p}$.

example: find $7^{222} \pmod{11}$. since 11 is prime, we know $7^{10} \equiv 1 \pmod{11}$. we can decompose the exponent 222: $222 = 10(22) + 2$.

$$\begin{aligned} 7^{222} &= (7^{10})^{22} \cdot 7^2 \\ &\equiv (1)^{22} \cdot 49 \pmod{11} \\ &\equiv 1 \cdot 5 \pmod{11} \\ &\equiv 5 \pmod{11} \end{aligned}$$

1.5 discrete logarithm

given a prime p and a primitive root r , for any integer b , solving for e in the equation:

$$r^e \equiv b \pmod{p}$$

is called the **discrete logarithm problem**. unlike standard logs, there is no known polynomial time algorithm to solve this efficiently. this hardness is the basis of many cryptographic systems!